DESARROLLO WEB INTEGRADO · 100000ST61

Laboratorio Guiado — Sesión 8

Seguridad con Spring Security: Roles y Permisos

Caso del laboratorio

Construirás, paso a paso, un Sistema de Registro de Notas de Alumnos con dos roles: DOCENTE (registra, modifica y elimina notas) y ALUMNO (solo consulta sus propias notas, una vez registradas).

Stack tecnológico: Java 21 · IntelliJ IDEA · Maven · Spring Boot 4 · Spring Security · Spring Data JPA · MySQL · Postman

1. Objetivos de aprendizaje

Al finalizar este laboratorio, serás capaz de:

- Configurar Spring Security en un proyecto Spring Boot 4 desde cero.
- Definir usuarios y roles persistidos en base de datos (no en memoria).
- Cifrar contraseñas correctamente con BCryptPasswordEncoder.
- Proteger endpoints REST según el rol del usuario autenticado, usando SecurityFilterChain.
- Aplicar autorización a nivel de dato: asegurar que un ALUMNO solo pueda ver SUS PROPIAS notas, no las de otros.
- Probar toda la seguridad configurada con Postman, incluyendo casos de éxito y de rechazo (401/403).

Por qué este laboratorio importa

El patrón que vas a construir aquí —usuarios con roles en base de datos, contraseñas cifradas, y endpoints protegidos por rol— es exactamente el que usarás en tu Proyecto Integrador y en cualquier proyecto profesional con Spring Boot. Repasa cada paso con calma: lo vas a replicar muchas veces.

2. El caso: Registro de Notas de Alumnos

Vamos a construir una pequeña API REST para que un centro de estudios registre y consulte notas. El sistema define dos roles con capacidades muy distintas:

Rol	Puede hacer	No puede hacer
DOCENTE	Registrar, modificar y eliminar notas de cualquier alumno. Consultar todas las notas.	No aplica restricciones adicionales en este laboratorio.
ALUMNO	Consultar únicamente SUS PROPIAS notas, una vez registradas por el docente.	Registrar, modificar o eliminar notas. Consultar notas de otro alumno.

2.1. Entidades del sistema

El modelo de datos tiene tres entidades principales:

- Usuario: representa la cuenta de acceso (username, password cifrada, rol). Es la entidad que Spring Security usará para autenticar.
- Alumno: datos académicos del estudiante (código, nombres, apellidos), vinculado 1 a 1 con un Usuario que tiene rol ALUMNO.
- Nota: una calificación de un alumno en un curso, vinculada a un Alumno (relación muchos a uno).

¿Por qué separar Usuario de Alumno?

Podríamos poner username/password directamente en Alumno, pero separar las cuentas de acceso (Usuario) de los datos de negocio (Alumno) es una buena práctica: el DOCENTE también es un Usuario (con rol DOCENTE), pero NO es un Alumno. Esta separación es la misma que usarás en cualquier proyecto con más de un tipo de usuario.

3. Requisitos previos

- JDK 21 instalado y configurado (java -version debe mostrar 21.x).
- IntelliJ IDEA (Community o Ultimate) con el plugin de Spring Boot.
- Maven (incluido en IntelliJ; no necesitas instalarlo aparte).
- MySQL Server 8.x en ejecución, y MySQL Workbench (o similar) para verificar datos.
- Postman instalado, para probar los endpoints protegidos.
- Haber completado la Sesión 6 (JPA/Hibernate) y la Sesión 7 (JPQL y Transacciones): usaremos entidades, repositorios y @Transactional con la misma sintaxis ya vista.

4. Parte 1 — Crear el proyecto en Spring Initializr

1. Abre https://start.spring.io (o usa el asistente integrado de IntelliJ IDEA: File → New → Project → Spring Initializr).
2. Configura los datos base del proyecto:

Campo	Valor
Project	Maven
Language	Java
Spring Boot	4.0.x (la versión estable más reciente)
Group	pe.utp
Artifact / Name	notas-alumnos-app
Package name	pe.utp.notasapp
Packaging	Jar
Java	21

3. Agrega las siguientes dependencias:

- Spring Web — para exponer los endpoints REST.
- Spring Data JPA — para persistencia con Hibernate.
- MySQL Driver — conector JDBC para MySQL.
- Spring Security — el framework que configuraremos en este laboratorio.
- Validation — para validar los datos que lleguen en los requests.

4. Genera el proyecto (Generate), descomprime el .zip y ábrelo en IntelliJ IDEA como proyecto Maven.
5. Espera a que Maven descargue todas las dependencias (barra de progreso inferior). Verifica que el proyecto compile sin errores antes de continuar.

Importante: apenas agregues Spring Security…

En cuanto termine de descargar las dependencias, si ejecutas la aplicación ahora mismo, TODOS los endpoints quedarán bloqueados automáticamente detrás de un formulario de login por defecto. Esto es normal: es la autoconfiguración de Spring Security. La vamos a personalizar por completo en la Parte 6.

5. Parte 2 — Configurar la base de datos

1. En MySQL Workbench (o consola MySQL), crea la base de datos:

```
CREATE DATABASE notas_alumnos_db;
```

2. Abre src/main/resources/application.properties y configura la conexión:

```
spring.application.name=notas-alumnos-app

# Conexión a MySQL
spring.datasource.url=jdbc:mysql://localhost:3306/notas_alumnos_db
spring.datasource.username=root
spring.datasource.password=TU_PASSWORD_AQUI

# Hibernate genera/actualiza las tablas automáticamente
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Reemplaza TU_PASSWORD_AQUI por la contraseña real de tu usuario root de MySQL. Con ddl-auto=update, Hibernate creará las tablas automáticamente a partir de las entidades que definiremos a continuación — no necesitas escribir el CREATE TABLE a mano.

6. Parte 3 — Modelo de datos: entidades y repositorios

6.1. Entidad Usuario

Esta es la entidad clave para Spring Security: contiene las credenciales de acceso y el rol.

Archivo: src/main/java/pe/utp/notasapp/model/Usuario.java

```
package pe.utp.notasapp.model;

import jakarta.persistence.*;
import com.fasterxml.jackson.annotation.JsonIgnore;

@Entity
@Table(name = "usuarios")
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String username;

    @JsonIgnore   // nunca debe salir en una respuesta JSON
    @Column(nullable = false)
    private String password;   // se guardará cifrada con BCrypt

    @Column(nullable = false)
    private String rol;        // "DOCENTE" o "ALUMNO"

    public Usuario() {}

    // --- Getters y Setters ---
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }

    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }

    public String getRol() { return rol; }
    public void setRol(String rol) { this.rol = rol; }
}
```

Detalle de seguridad que se pasa por alto fácilmente

@JsonIgnore en el campo password evita que el hash de la contraseña viaje en las respuestas JSON cuando un Alumno (y su Usuario) se serialicen como parte de una Nota. Nunca expongas contraseñas —ni siquiera cifradas— en una respuesta de API.

6.2. Entidad Alumno

Archivo: src/main/java/pe/utp/notasapp/model/Alumno.java

```
package pe.utp.notasapp.model;

import jakarta.persistence.*;

@Entity
@Table(name = "alumnos")
public class Alumno {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String codigo;

    private String nombres;
    private String apellidos;

    // Vínculo 1 a 1 con la cuenta de acceso del alumno
    @OneToOne
    @JoinColumn(name = "usuario_id", unique = true)
    private Usuario usuario;

    public Alumno() {}

    // --- Getters y Setters ---
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getCodigo() { return codigo; }
    public void setCodigo(String codigo) { this.codigo = codigo; }

    public String getNombres() { return nombres; }
    public void setNombres(String nombres) { this.nombres = nombres; }

    public String getApellidos() { return apellidos; }
    public void setApellidos(String apellidos) { this.apellidos = apellidos; }

    public Usuario getUsuario() { return usuario; }
    public void setUsuario(Usuario usuario) { this.usuario = usuario; }
}
```

6.3. Entidad Nota

Archivo: src/main/java/pe/utp/notasapp/model/Nota.java

```
package pe.utp.notasapp.model;

import jakarta.persistence.*;

@Entity
@Table(name = "notas")
public class Nota {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String curso;
    private Double calificacion;

    @ManyToOne
    @JoinColumn(name = "alumno_id", nullable = false)
    private Alumno alumno;

    public Nota() {}

    // --- Getters y Setters ---
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getCurso() { return curso; }
    public void setCurso(String curso) { this.curso = curso; }

    public Double getCalificacion() { return calificacion; }
    public void setCalificacion(Double calificacion) {
        this.calificacion = calificacion;
    }

    public Alumno getAlumno() { return alumno; }
    public void setAlumno(Alumno alumno) { this.alumno = alumno; }
}
```

6.4. Repositorios

Los tres repositorios siguen el mismo patrón que ya usaste en la Sesión 6 con JpaRepository.

Archivo: repository/UsuarioRepository.java

```
package pe.utp.notasapp.repository;

import pe.utp.notasapp.model.Usuario;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
    Optional<Usuario> findByUsername(String username);
}
```

Archivo: repository/AlumnoRepository.java

```
package pe.utp.notasapp.repository;

import pe.utp.notasapp.model.Alumno;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface AlumnoRepository extends JpaRepository<Alumno, Long> {
    Optional<Alumno> findByUsuario_Username(String username);
}
```

Archivo: repository/NotaRepository.java

```
package pe.utp.notasapp.repository;

import pe.utp.notasapp.model.Nota;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface NotaRepository extends JpaRepository<Nota, Long> {
    List<Nota> findByAlumno_Id(Long alumnoId);
}
```

findByUsuario_Username y findByAlumno_Id son métodos derivados: Spring Data los interpreta navegando la relación (Alumno.usuario.username y Nota.alumno.id) sin que escribas ni una línea de JPQL. Si necesitaras algo más complejo, recuerda que en la Sesión 7 aprendiste a usar @Query para eso.

7. Parte 4 — Cifrado de contraseñas

Antes de conectar nada con Spring Security, creamos el bean que cifrará (y luego verificará) las contraseñas. Nunca guardaremos contraseñas en texto plano.

Archivo: src/main/java/pe/utp/notasapp/config/SecurityConfig.java (lo iremos completando en varias partes)

```
package pe.utp.notasapp.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

¿Por qué BCrypt y no {noop}?

BCrypt aplica un hash de una sola vía más una "sal" aleatoria: la contraseña original nunca queda expuesta, ni siquiera para quien tiene acceso a la base de datos. {noop} (texto plano) solo sirve para pruebas rápidas y nunca debe usarse en un proyecto real. Este bean es el único lugar del proyecto donde definimos el algoritmo; el resto del sistema simplemente lo inyecta donde lo necesite.

8. Parte 5 — Conectar Usuario con Spring Security

Spring Security no sabe nada de nuestra entidad Usuario todavía. El puente entre ambos es una clase que implementa UserDetailsService: cada vez que alguien intenta iniciar sesión, Spring Security la invoca automáticamente para buscar al usuario por su username.

Archivo: src/main/java/pe/utp/notasapp/security/UsuarioDetailsService.java

```
package pe.utp.notasapp.security;

import pe.utp.notasapp.model.Usuario;
import pe.utp.notasapp.repository.UsuarioRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.userdetails.*;
import org.springframework.stereotype.Service;

@Service
public class UsuarioDetailsService implements UserDetailsService {

    @Autowired
    private UsuarioRepository usuarioRepository;

    @Override
    public UserDetails loadUserByUsername(String username) {
        Usuario u = usuarioRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException(
                "Usuario no encontrado: " + username));

        return User.builder()
```

```
            .username(u.getUsername())
            .password(u.getPassword())   // ya viene cifrada con BCrypt
            .roles(u.getRol())            // "DOCENTE" o "ALUMNO"
            .build();
    }
}
```

roles(u.getRol()) es la línea más importante de esta clase: toma el texto guardado en la base de datos ("DOCENTE" o "ALUMNO") y Spring Security internamente lo convierte en la autoridad ROLE_DOCENTE o ROLE_ALUMNO. Ese es el mismo mecanismo que viste en la sesión de teoría.

9. Parte 6 — SecurityFilterChain: la configuración central

Ahora sí, el corazón del laboratorio: el bean que define QUÉ rutas son públicas y QUIÉN puede acceder a cada una. Completa la clase SecurityConfig así:

Archivo: config/SecurityConfig.java (versión completa)

```
package pe.utp.notasapp.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

import static org.springframework.http.HttpMethod.*;

@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http)
            throws Exception {

        http.authorizeHttpRequests(auth -> auth
            // Regla específica PRIMERO: solo el DOCENTE lista TODAS
            .requestMatchers(GET, "/api/notas/todas")
                .hasRole("DOCENTE")
            // El alumno consulta su historial (el docente también)
            .requestMatchers(GET, "/api/notas/mis-notas")
                .hasAnyRole("DOCENTE", "ALUMNO")
            // Registrar, modificar y eliminar notas: solo DOCENTE
            .requestMatchers(POST, "/api/notas/**")
                .hasRole("DOCENTE")
            .requestMatchers(PUT, "/api/notas/**")
                .hasRole("DOCENTE")
            .requestMatchers(DELETE, "/api/notas/**")
                .hasRole("DOCENTE")
            // Cualquier otra ruta: exige, al menos, estar autenticado
            .anyRequest().authenticated()
        )
        .httpBasic(Customizer.withDefaults())  // login vía Postman
        .csrf(csrf -> csrf.disable());         // API sin estado

        return http.build();
    }
}
```

9.1. Por qué cada regla está en ese orden

Spring Security evalúa estas reglas de arriba hacia abajo y aplica la PRIMERA que coincida con la petición. Por eso el orden importa:

Regla	Explicación
GET /api/notas/todas → hasRole("DOCENTE")	Va PRIMERO por ser la más específica. Si estuviera después de una regla genérica "/api/notas/**", nunca se aplicaría (Spring Security se queda con la primera coincidencia).
GET /api/notas/mis-notas → hasAnyRole("DOCENTE","ALUMNO")	Ambos roles pueden entrar aquí. Dentro del controlador (Parte 7) filtraremos para que cada quien vea solo lo que le corresponde: eso Spring Security no lo resuelve solo, es autorización a nivel de dato.
POST/PUT/DELETE /api/notas/** → hasRole("DOCENTE")	Solo el DOCENTE puede escribir. Un ALUMNO autenticado que intente estos métodos recibirá 403 Forbidden.
anyRequest().authenticated()	Regla de respaldo: cualquier otra ruta no listada arriba exige, como mínimo, haber iniciado sesión.

Regla de oro: lo específico va antes que lo general

/api/notas/todas y /api/notas/mis-notas también coinciden con el patrón /api/notas/**. Si hubiéramos puesto una regla genérica para /api/notas/** antes que estas dos, esa regla genérica habría "capturado" ambas rutas primero y las reglas específicas jamás se habrían aplicado. Por eso: primero las rutas más específicas, al final la más general.

httpBasic() en vez de formLogin()

Como este laboratorio expone una API REST (no páginas HTML), usamos autenticación HTTP Basic: el cliente (Postman) envía username y password en cada petición, en el encabezado Authorization. Es el mismo mecanismo que probaste en la teoría con httpBasic(Customizer.withDefaults()).

10. Parte 7 — Controlador de Notas: roles + autorización a nivel de dato

Con SecurityFilterChain ya controlamos QUIÉN puede llamar a cada endpoint según su rol. Pero falta un segundo nivel: dentro de /api/notas/mis-notas, un ALUMNO debe ver únicamente SUS propias notas, no las de sus compañeros. Eso no lo resuelve el rol — lo resuelve tu código, comparando el usuario autenticado con el dueño del dato.

Archivo: src/main/java/pe/utp/notasapp/controller/NotaController.java

```
package pe.utp.notasapp.controller;

import pe.utp.notasapp.model.Alumno;
import pe.utp.notasapp.model.Nota;
import pe.utp.notasapp.repository.AlumnoRepository;
import pe.utp.notasapp.repository.NotaRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/notas")
public class NotaController {

    @Autowired
    private NotaRepository notaRepository;

    @Autowired
    private AlumnoRepository alumnoRepository;

    // Solo DOCENTE llega aquí (protegido en SecurityConfig)
    @GetMapping("/todas")
    public List<Nota> listarTodas() {
        return notaRepository.findAll();
    }

    // DOCENTE y ALUMNO llegan aquí; filtramos según quién pregunta
    @GetMapping("/mis-notas")
    public ResponseEntity<?> misNotas(Authentication authentication) {
        String username = authentication.getName();

        Alumno alumno = alumnoRepository.findByUsuario_Username(username)
            .orElse(null);

        if (alumno == null) {
            // Quien llama es un DOCENTE (no tiene registro de Alumno)
            return ResponseEntity.badRequest()
                .body("Este usuario no tiene un perfil de alumno asociado.");
        }

        List<Nota> notas = notaRepository.findByAlumno_Id(alumno.getId());
        return ResponseEntity.ok(notas);
    }

    // Solo DOCENTE llega aquí (protegido en SecurityConfig)
    @PostMapping
    public ResponseEntity<Nota> registrar(@RequestBody Nota nota) {
        Nota guardada = notaRepository.save(nota);
        return ResponseEntity.ok(guardada);
    }
```

```
    // Solo DOCENTE llega aquí (protegido en SecurityConfig)
    @PutMapping("/{id}")
    public ResponseEntity<?> modificar(
            @PathVariable Long id, @RequestBody Nota datos) {
        return notaRepository.findById(id).map(nota -> {
            nota.setCurso(datos.getCurso());
            nota.setCalificacion(datos.getCalificacion());
            notaRepository.save(nota);
            return ResponseEntity.ok(nota);
        }).orElse(ResponseEntity.notFound().build());
    }

    // Solo DOCENTE llega aquí (protegido en SecurityConfig)
    @DeleteMapping("/{id}")
    public ResponseEntity<?> eliminar(@PathVariable Long id) {
        if (!notaRepository.existsById(id)) {
            return ResponseEntity.notFound().build();
        }
        notaRepository.deleteById(id);
        return ResponseEntity.noContent().build();
    }
}
```

La línea que hace la diferencia: authentication.getName()

Spring Security inyecta automáticamente el objeto Authentication en cualquier método del controlador que lo declare como parámetro. authentication.getName() devuelve el username del usuario que inició sesión en ESTA petición. Con eso buscamos su Alumno asociado y filtramos las notas por alumno.getId() — así, aunque un ALUMNO conozca el endpoint, nunca puede ver ni manipular datos de otro alumno con este diseño.

Para replicar en tus propios proyectos

Este patrón (regla de rol en SecurityConfig + filtro por dueño del dato con Authentication en el controlador) es la forma correcta de combinar "seguridad por rol" con "seguridad por propietario". Úsalo cada vez que un usuario deba ver solo SUS PROPIOS registros: sus pedidos, su carrito, sus reservas, etc.

11. Parte 8 — Cargar datos iniciales para probar

Necesitamos al menos un DOCENTE y dos ALUMNOS para probar la seguridad. Los creamos con un CommandLineRunner, que se ejecuta una sola vez al iniciar la aplicación — así las contraseñas se cifran con el mismo BCryptPasswordEncoder que ya configuramos, sin tener que calcular el hash a mano.

Archivo: src/main/java/pe/utp/notasapp/config/DataLoader.java

```
package pe.utp.notasapp.config;

import pe.utp.notasapp.model.Alumno;
import pe.utp.notasapp.model.Usuario;
import pe.utp.notasapp.repository.AlumnoRepository;
import pe.utp.notasapp.repository.UsuarioRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Component;

@Component
public class DataLoader implements CommandLineRunner {

    private final UsuarioRepository usuarioRepository;
    private final AlumnoRepository alumnoRepository;
    private final PasswordEncoder passwordEncoder;

    public DataLoader(UsuarioRepository usuarioRepository,
                      AlumnoRepository alumnoRepository,
                      PasswordEncoder passwordEncoder) {
        this.usuarioRepository = usuarioRepository;
        this.alumnoRepository = alumnoRepository;
        this.passwordEncoder = passwordEncoder;
    }

    @Override
    public void run(String... args) {
        if (usuarioRepository.count() > 0) return; // evita duplicar al reiniciar

        // --- Docente ---
        Usuario docente = new Usuario();
        docente.setUsername("docente1");
        docente.setPassword(passwordEncoder.encode("docente123"));
        docente.setRol("DOCENTE");
        usuarioRepository.save(docente);

        // --- Alumno 1: Ana ---
        Usuario userAna = new Usuario();
        userAna.setUsername("ana01");
        userAna.setPassword(passwordEncoder.encode("ana123"));
        userAna.setRol("ALUMNO");
        usuarioRepository.save(userAna);

        Alumno ana = new Alumno();
        ana.setCodigo("A001");
        ana.setNombres("Ana");
        ana.setApellidos("Torres");
        ana.setUsuario(userAna);
        alumnoRepository.save(ana);

        // --- Alumno 2: Luis ---
        Usuario userLuis = new Usuario();
        userLuis.setUsername("luis02");
        userLuis.setPassword(passwordEncoder.encode("luis123"));
        userLuis.setRol("ALUMNO");
        usuarioRepository.save(userLuis);

        Alumno luis = new Alumno();
```

```
        luis.setCodigo("A002");
        luis.setNombres("Luis");
        luis.setApellidos("Ramírez");
        luis.setUsuario(userLuis);
        alumnoRepository.save(luis);

        System.out.println(
            ">> Datos iniciales cargados: docente1 / ana01 / luis02");
    }
}
```

Ejecuta la aplicación (Run). En la consola deberías ver el mensaje de confirmación, y en MySQL Workbench podrás revisar las tablas usuarios y alumnos ya pobladas.

Usuario	Contraseña	Rol
docente1	docente123	DOCENTE
ana01	ana123	ALUMNO (Ana Torres, código A001)
luis02	luis123	ALUMNO (Luis Ramírez, código A002)

12. Parte 9 — Probar la seguridad con Postman

Vamos a verificar, uno por uno, que las reglas de autorización realmente funcionan — incluyendo los casos donde DEBE fallar.

12.1. Configurar autenticación Basic en Postman

1. Crea una nueva request en Postman (por ejemplo, POST http://localhost:8080/api/notas).
2. Ve a la pestaña Authorization.
3. En Auth Type, selecciona Basic Auth.
4. Ingresa el Username y Password del usuario con el que quieras probar (ej. docente1 / docente123).
5. Postman codificará automáticamente las credenciales en el encabezado Authorization en cada petición.

12.2. Escenarios de prueba

Ejecuta estos escenarios EN ORDEN y anota el resultado real que obtuviste junto al esperado:

#	Escenario	Método y URL	Resultado esperado
1	docente1 registra una nota para Ana (alumno_id=1)	POST /api/notas Body: {"curso":"Desarrollo Web","calificacion":16,"alumno":{"id":1}}	**200 OK — nota creada**
2	docente1 lista todas las notas	GET /api/notas/todas	**200 OK — lista completa**
3	ana01 intenta registrar una nota	POST /api/notas	**403 Forbidden**
4	ana01 consulta sus propias notas	GET /api/notas/mis-notas	**200 OK — solo las notas de Ana**
5	ana01 intenta listar TODAS las notas	GET /api/notas/todas	**403 Forbidden**
6	luis02 consulta "mis-notas"	GET /api/notas/mis-notas	**200 OK — lista vacía o solo sus notas (NUNCA las de Ana)**
7	Sin credenciales, cualquier endpoint	GET /api/notas/mis-notas (sin Auth)	**401 Unauthorized**
8	docente1 elimina la nota creada en el paso 1	DELETE /api/notas/1	**204 No Content**

El escenario más importante: #6

El paso 6 es la prueba real de que tu autorización a nivel de dato funciona. Si luis02 pudiera ver las notas de Ana, significaría que el filtro por alumno.getId() en el controlador no está funcionando, aunque los roles estén perfectamente configurados. Roles correctos + filtro de dueño incorrecto sigue siendo una falla de seguridad.

Diferencia clave entre los pasos 3/5 y el paso 7

En los pasos 3 y 5, el usuario SÍ se autenticó (Postman envió credenciales válidas), pero su rol no tiene permiso → 403 Forbidden. En el paso 7 no hay credenciales en absoluto → 401 Unauthorized. Es la misma distinción "autenticación vs. autorización" que viste en la teoría, ahora comprobada con tus propios ojos.

13. Reto avanzado (opcional)

Si terminaste antes de tiempo, extiende el laboratorio con este reto: agrega un endpoint para que el DOCENTE consulte las notas de un alumno específico por su id, protegido además a nivel de método con @PreAuthorize.

1. Habilita la seguridad a nivel de método agregando @EnableMethodSecurity a la clase SecurityConfig.
2. Agrega este método al NotaController:

```
@PreAuthorize("hasRole('DOCENTE')")
@GetMapping("/alumno/{alumnoId}")
public List<Nota> notasDeAlumno(@PathVariable Long alumnoId) {
    return notaRepository.findByAlumno_Id(alumnoId);
}
```

3. Prueba en Postman: docente1 debería poder consultar GET /api/notas/alumno/1 (notas de Ana) sin problema.
4. Ahora intenta lo mismo con ana01 → debe recibir 403, aunque la ruta no esté explícitamente listada en el SecurityFilterChain (cae en anyRequest().authenticated(), pero @PreAuthorize la bloquea igual por rol).

Para pensar

¿Qué pasaría si, en vez de hasRole('DOCENTE'), quisieras permitir también que CADA alumno consulte SU PROPIO id por esta misma ruta? Pista: @PreAuthorize acepta expresiones más complejas, evaluando parámetros del método (ej. comparando alumnoId contra el alumno autenticado). Ese es el mismo principio de "autorización a nivel de dato" que ya aplicaste en /mis-notas, pero expresado como anotación en vez de código dentro del método.

14. Preguntas de reflexión

Respóndelas con tus propias palabras, basándote en lo que construiste (no en la teoría memorizada):

1. ¿Por qué separamos la entidad Usuario (credenciales) de la entidad Alumno (datos académicos), en vez de poner username/password directamente en Alumno?
2. En el endpoint /api/notas/mis-notas, explica con tus palabras por qué hasAnyRole("DOCENTE","ALUMNO") en el SecurityFilterChain NO es suficiente por sí solo para garantizar que un alumno no vea las notas de otro.
3. ¿Qué pasaría si, por error, hubieras colocado la regla .requestMatchers("/api/notas/**").hasAnyRole("DOCENTE","ALUMNO") ANTES que la regla específica para /api/notas/todas? Describe el síntoma que verías al probarlo.
4. ¿Por qué usamos @JsonIgnore en el campo password de Usuario? ¿Qué problema de seguridad concreto evita?
5. Si tuvieras que agregar un tercer rol, por ejemplo COORDINADOR (que puede ver todas las notas pero no modificarlas), ¿qué cambiarías exactamente en el SecurityFilterChain? Descríbelo en pseudocódigo.

15. Rúbrica de evaluación

Puntaje total: 20 puntos.

Criterio	Descripción	Puntos
Configuración del proyecto	Proyecto creado correctamente con Java 21, Spring Boot 4 y las dependencias indicadas; conexión a MySQL funcionando.	2
Modelo de datos	Entidades Usuario, Alumno y Nota correctamente mapeadas, con las relaciones @OneToOne y @ManyToOne funcionando.	3
Cifrado de contraseñas	PasswordEncoder con BCrypt configurado y usado correctamente al crear los usuarios iniciales.	2
UserDetailsService	Implementación correcta que conecta la entidad Usuario con la autenticación de Spring Security.	3
SecurityFilterChain	Reglas de autorización por rol correctamente definidas, en el orden correcto (específicas antes que generales).	4
Autorización a nivel de dato	El endpoint /mis-notas filtra correctamente por el usuario autenticado; un alumno nunca ve notas de otro.	4
Pruebas en Postman	Los 8 escenarios de prueba fueron ejecutados y los resultados coinciden con lo esperado (incluyendo 401 y 403).	2

El reto avanzado (sección 13) y las preguntas de reflexión (sección 14) no suman puntos adicionales a esta rúbrica, pero se recomienda completarlos: son la base de las preguntas conceptuales de la próxima evaluación.

16. Recursos y referencias

- Documentación oficial de Spring Security — Authorize HTTP Requests.
- Documentación oficial de Spring Security — Password Storage (BCrypt).
- Material de la Sesión 6 (JPA / Hibernate / CRUD) para repasar el mapeo de relaciones @OneToOne y @ManyToOne.
- Diapositivas de la Sesión 8: Seguridad con Spring Security — Configuración de roles y permisos.

¡Éxitos con tu laboratorio! Cualquier duda, participa en el foro del curso.